

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Фронт-энд разработка

Отчет

Лабораторная работа № 2

Выполнил:

Бессонов Александр
J3213

Проверил:

Добряков Д. И.

Санкт-Петербург
2026 г.

Задача

Связать интерфейс, разработанный в лабораторной работе № 1, с внешним API средствами `fetch`, `axios` или `XMLHttpRequest`. Реализовать моковый REST API на базе JSON Server и добавить авторизацию по образцу рассмотренных на занятиях решений.

Выбранный вариант

Вариант 9 - сервис для управления проектами и командной работы. В приложении данные задач, проектов, участников и комментариев получаются из мокового REST API. Вход и регистрация выполняются через защищённые маршруты API.

Ход работы

1. Настройка мокового сервера. В проект добавлены JSON Server и `json-server-auth`. Файл `server.js` публикует статические страницы и REST-маршруты на порту 3000, а `db.json` хранит пользователей, рабочие пространства, проекты, задачи, участников и комментарии.

2. Реализация авторизации. Форма входа отправляет POST-запрос на `/login`. Полученный токен и данные пользователя сохраняются в `localStorage` либо `sessionStorage`, после чего токен передаётся в заголовке `Authorization` всех последующих запросов.

3. Создание API-клиента. В `api.js` создан единый асинхронный метод `request`. Он формирует заголовки, вызывает `fetch`, разбирает JSON-ответы и преобразует ошибки сервера в понятные сообщения интерфейса.

4. Подключение интерфейса к данным. Задачи, участники и комментарии загружаются GET-запросами. Создание задачи выполняется POST-запросом, а перенос карточки и изменение роли участника сохраняются PATCH-запросами.

5. Проверка результата. Проверены успешная авторизация, загрузка коллекций, создание задачи и изменение её статуса. Сервер вернул коды 200 и 201, а интерфейс обновил данные без перезагрузки всей страницы.

Структура проекта

<code>server.js</code>	- запуск JSON Server и модуля авторизации
<code>db.json</code>	- данные мокового REST API
<code>assets/js/api.js</code>	- fetch-клиент и методы работы с API
<code>assets/js/app.js</code>	- обработчики интерфейса и загрузка данных
<code>index.html</code>	- форма входа
<code>register.html</code>	- регистрация пользователя и пространства
<code>project.html</code>	- доска задач и комментарии
<code>backlog.html</code>	- создание задач
<code>search.html</code>	- поиск загруженных задач
<code>team.html</code>	- участники и роли

Листинг 1 - Основные файлы лабораторной работы

Моковый REST API

Сервер создаётся средствами json-server и использует db.json как базу данных. Middleware json-server-auth добавляет маршруты /login и /register, проверяет токен и применяет правила доступа. Статические HTML-, CSS- и JavaScript-файлы обслуживаются тем же процессом на порту 3000.

```
const jsonServer = require('json-server');
const auth = require('json-server-auth');

const server = jsonServer.create();
const router = jsonServer.router('db.json');
server.db = router.db;
server.use(jsonServer.defaults({ static: __dirname, logger: true }));
server.use(jsonServer.bodyParser);
server.use(auth.rewriter({ users: 600 }));
server.use(auth);
server.use(router);
server.listen(3000);
```

Листинг 2 - Настройка JSON Server и авторизации

Авторизация пользователя

Метод login отправляет электронную почту и пароль в JSON-формате. После ответа сервера accessToken и объект пользователя сохраняются в выбранном хранилище браузера. Если установлен флаг “Оставаться в системе”, используется localStorage, иначе данные действуют только в текущей вкладке через sessionStorage.

```
async function login(email, password, remember) {
  const payload = await request('/login', {
    method: 'POST',
    body: JSON.stringify({ email, password })
  });
  saveSession(payload, remember);
  return payload.user;
}

function saveSession(payload, remember = false) {
  const storage = remember ? localStorage : sessionStorage;
  clearSession();
  storage.setItem(tokenKey, payload.accessToken);
  storage.setItem(userKey, JSON.stringify(payload.user));
}
```

Листинг 3 - Вход и сохранение пользовательской сессии

Единый клиент API

Функция `request` используется всеми методами API. Она автоматически задаёт `Content-Type` для JSON-тела, добавляет Bearer-токен, обрабатывает сетевую ошибку и проверяет HTTP-статус ответа. Благодаря этому обработчики интерфейса работают с единым форматом данных и ошибок.

```
async function request(path, options = {}) {
  const headers = new Headers(options.headers || {});
  const token = getToken();
  if (options.body) headers.set('Content-Type', 'application/json');
  if (token) headers.set('Authorization', `Bearer ${token}`);

  const response = await fetch(apiOrigin + path, { ...options, headers });
  const data = await response.json();
  if (!response.ok) {
    throw new Error(data.message || `Ошибка API: ${response.status}`);
  }
  return data;
}
```

Листинг 4 - Выполнение авторизованного HTTP-запроса

Методы работы с ресурсами

Средство	Применение
POST /login	Проверка учётных данных и получение <code>accessToken</code> .
GET /tasks	Загрузка задач пользователя для доски, бэклога и поиска.
POST /tasks	Создание новой задачи с данными формы бэклога.
PATCH /tasks/:id	Сохранение ключа созданной задачи и нового статуса карточки.
GET /members	Загрузка участников выбранного рабочего пространства.
PATCH /members/:id	Изменение роли участника в интерфейсе команды.

Сохранение статуса задачи

После переноса карточки между колонками интерфейс сначала обновляет её положение, затем отправляет PATCH-запрос. При ошибке показанное изменение отменяется повторной загрузкой задач, поэтому состояние страницы остаётся согласованным с сервером.

```
kanbanBoard?.addEventListener('drop', async (event) => {
  const zone = event.target.closest('[data-dropzone]');
  const card = draggedKanbanCard;
  if (!zone || !card) return;
  event.preventDefault();
  zone.append(card);
  updateKanbanCounts();
  try {
    await api.updateTask(card.dataset.recordId,
      { status: zone.dataset.dropzone });
    showToast(`Статус задачи ${card.dataset.taskId} сохранён в API`);
  } catch (error) {
    await loadKanbanTasks();
  }
});
```

Листинг 5 - Обновление статуса задачи через PATCH-запрос

Авторизация через API

Форма отправляет введённые данные на маршрут /login. Сервер проверил учётные данные и вернул ответ 200, после чего пользователь получил доступ к защищённым страницам приложения.

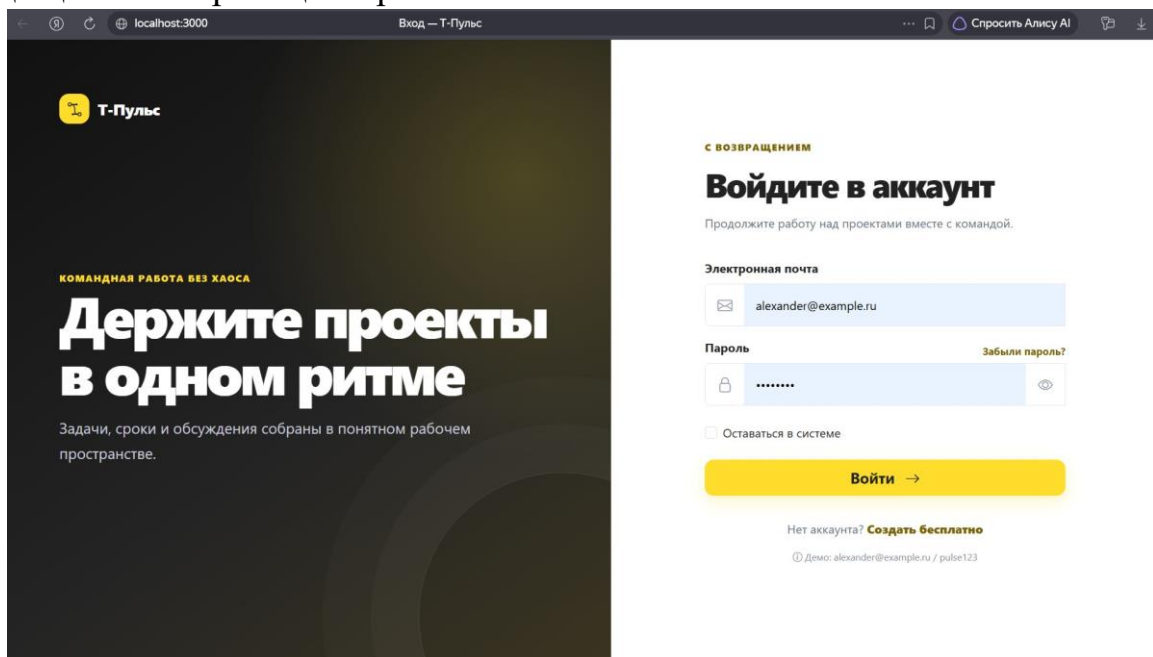


Рисунок 1 - Форма входа с демонстрационными данными

```
(base) PS C:\projects\frontend\ITMO-AIT-Frontend-2026\J3213\Бессонов Александр\labs\lab2> npm install
>>
npm notice Changelog: https://github.com/npm/cli/releases/tag/v11.19.1
npm notice To update run: npm install -g npm@11.19.1
npm notice
(base) PS C:\projects\frontend\ITMO-AIT-Frontend-2026\J3213\Бессонов Александр\labs\lab2>
❖ (base) PS C:\projects\frontend\ITMO-AIT-Frontend-2026\J3213\Бессонов Александр\labs\lab2> npm start

> t-pulse-lab2@1.0.0 start
> node server.js

Т-Пульс запущен: http://localhost:3000
REST API доступен: http://localhost:3000/tasks
POST /login 200 595.894 ms - 467
```

Рисунок 2 - Запуск сервера и успешный POST-запрос авторизации

Загрузка и изменение задач

После входа отображается личный кабинет пользователя. Доска проекта формируется из задач, полученных GET-запросом. Карточка TP-124 перенесена в колонку «В работе», а сервер сохранил новый статус запросом PATCH /tasks/124.

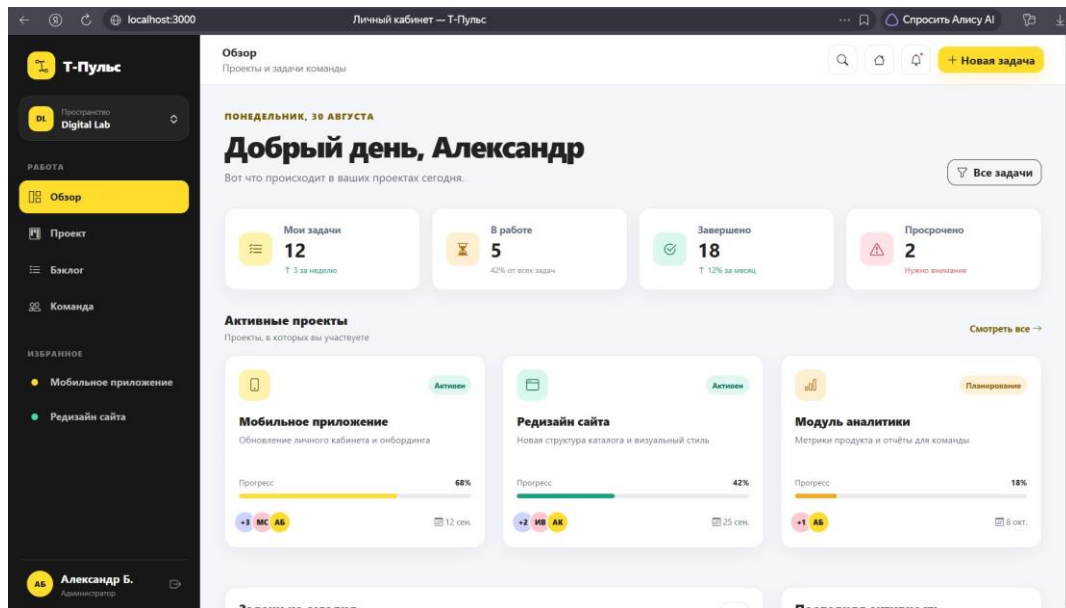


Рисунок 3 - Личный кабинет после авторизации

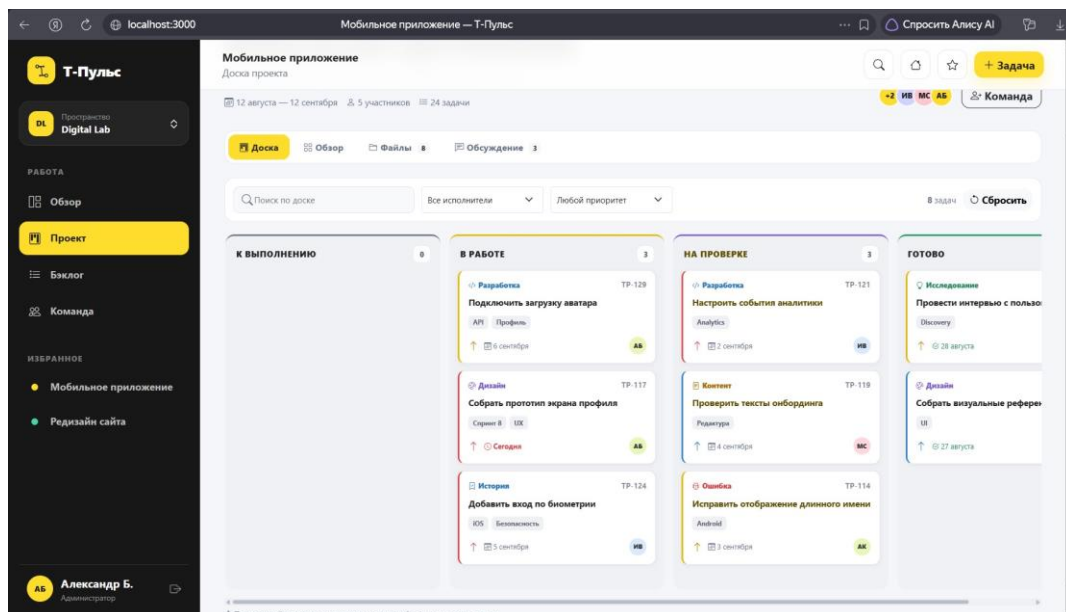


Рисунок 4 - Доска после изменения статуса задачи TP-124

```
T-Пульс запущен: http://localhost:3000
REST API доступен: http://localhost:3000/tasks
POST /login 200 595.894 ms - 467
GET /comments?projectId=1&sort=createdAt&order=asc 200 38.006 ms - 2
GET /tasks?userId=1&scope=board&sort=id&order=desc 200 36.127 ms - -
PATCH /tasks/124 200 34.026 ms - 430
```

Рисунок 5 - Подтверждение PATCH-запроса в журнале сервера

Создание задачи в бэклоге

Данные формы преобразуются в объект задачи и отправляются методом POST. После создания серверу передаётся PATCH-запрос для присвоения ключа TP-138, затем список загружается повторно и новая задача появляется в бэклоге.

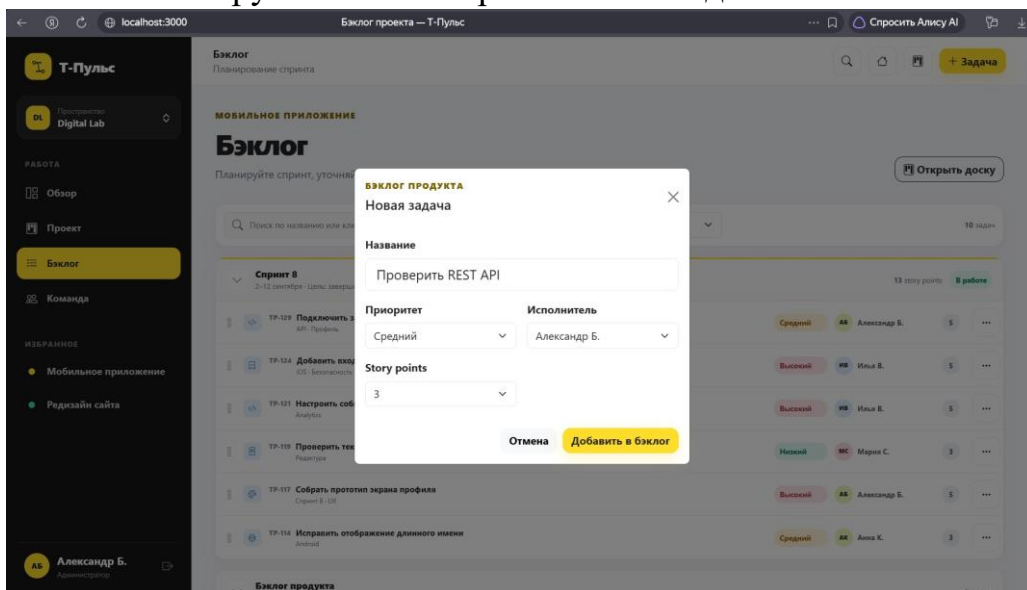


Рисунок 6 - Заполнение формы новой задачи

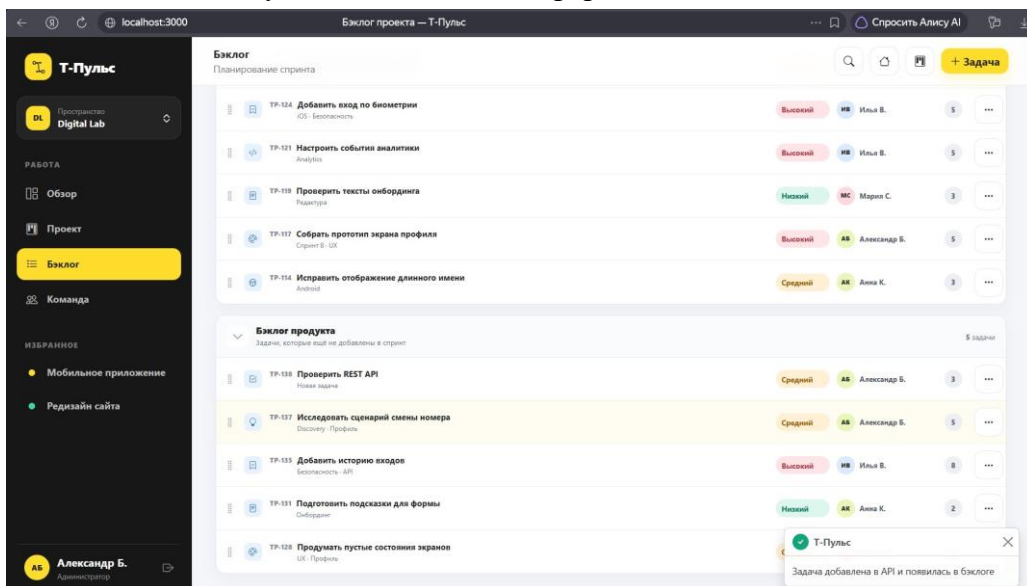


Рисунок 7 - Задача TP-138 после сохранения в API

```
T-Пульс запущен: http://localhost:3000
REST API доступен: http://localhost:3000/tasks
POST /login 200 595.894 ms - 467
GET /comments?projectId=1&_sort=createdAt&_order=asc 200 38.006 ms - 2
GET /tasks?userId=1&scope=board&_sort=id&_order=desc 200 36.127 ms - -
PATCH /tasks/124 200 34.026 ms - 430
GET /tasks?userId=1&_sort=id&_order=desc 200 26.763 ms - -
POST /tasks 201 34.317 ms - 387
PATCH /tasks/138 200 26.852 ms - 406
GET /tasks?userId=1&_sort=id&_order=desc 200 25.978 ms - -
```

Рисунок 8 - Запросы POST, PATCH и повторная загрузка задач

Поиск и участники команды

Поиск выполняется по задачам, загруженным из API, и немедленно обновляет список результатов. Раздел команды получает участников рабочего пространства запросом GET /members и выводит их роли и статусы.

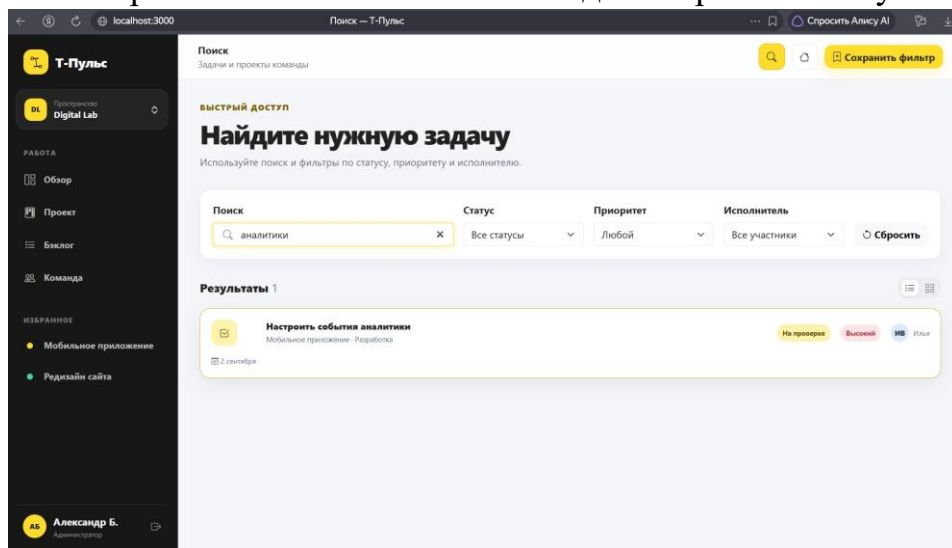


Рисунок 9 - Фильтрация задач, полученных из API

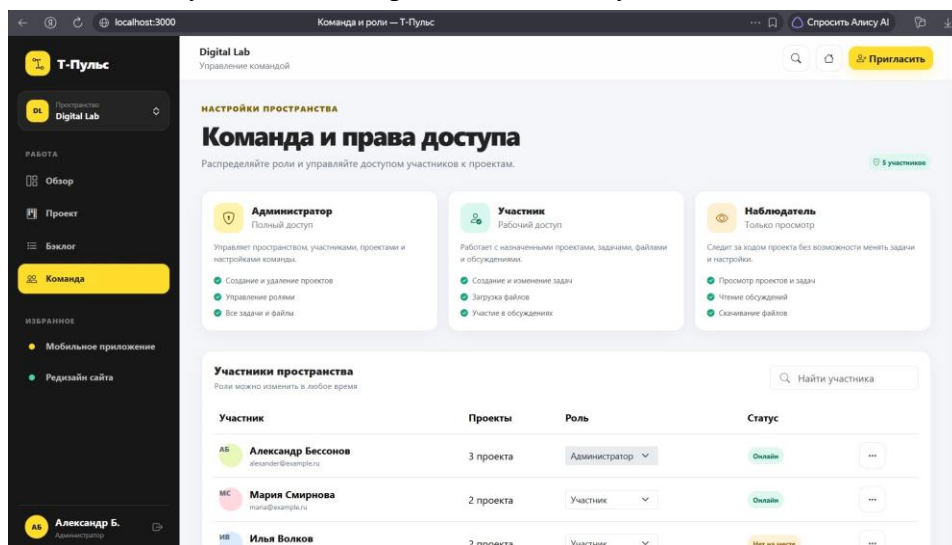


Рисунок 10 - Участники и роли рабочего пространства

```
T-Пульс запущен: http://localhost:3000
REST API доступен: http://localhost:3000/tasks
POST /login 200 595.894 ms - 467
GET /comments?projectId=1&sort=createdAt&order=asc 200 38.006 ms - 2
GET /tasks?userId=1&scope=board&sort=id&order=desc 200 36.127 ms - -
PATCH /tasks/124 200 34.026 ms - 430
GET /tasks?userId=1&sort=id&order=desc 200 26.763 ms - -
POST /tasks 201 34.317 ms - 387
PATCH /tasks/138 200 26.852 ms - 406
GET /tasks?userId=1&sort=id&order=desc 200 25.978 ms - -
GET /tasks?userId=1&sort=id&order=desc 304 26.624 ms - -
GET /members?workspaceId=1&sort=id&order=asc 200 16.494 ms - -
```

Рисунок 11 - GET-запрос списка участников

Вывод

Сервис “Т-Пульс” подключён к JSON Server. Авторизация и операции GET, POST и PATCH выполнены средствами Fetch API и подтверждены ответами сервера.